

PORTFOLIO

기획 → UI 설계 → 개발까지 전 과정을 수행하는
Front-End Developer 임나슬

Name : 임나슬 Lim Na Seul

E-mail : limnaseul52278@gmail.com

Phone Number : +82) 010-8923-1353

#web Design #UI/UX #Frontend #HTML #CSS #JavaScript

About me

안녕하세요.
임나슬입니다. :)



ABOUT ME

LIM NA SEUL / 임나슬 1992.05.22

UI/UX 기반으로 사용자 흐름을 설계하고, 데이터 흐름과 UI 상태를 연결하는 프론트엔드 개발을 지향합니다.

- ✓ 사용자 흐름 기반 UI 설계 + 데이터 구조 설계 경험 보유
- ✓ 지도 기반 서비스 및 상태 동기화 구조 설계 경험

CONTACT

✉ limnaseul52278@gmail.com

☎ +82) 010-8923-1353

EDUCATION

2011.02 보영여자 고등학교 졸업

2013.02 충청대학교 시각디자인과 졸업

SKILLS

Frontend

- HTML5 / SCSS / Bootstrap
- JavaScript (ES6+) / 데이터 기반 UI 렌더링 및 상태 제어
- React (상태 관리 및 컴포넌트 구조 학습 중)

Tools & Environment

- Git / GitHub
- Netlify (배포 경험)
- Figma (UI 설계)

EXPERIENCE

2014. 10 ~ 2015. 01	호텔박사
2015. 03 ~ 2016. 04	(주)마에스트로특별한여행
2016. 04 ~ 2018. 04	이케이맨파워(주)
2018. 04 ~ 2024. 02	잡코리아유한책임회사
2025. 05 ~ 현재	주식회사 윤슬

Core Strengths

① 사용자 흐름 기반 UI 설계 및 개선

- ✓ 사용자 행동 흐름을 기반으로 문제를 정의하고
- ✓ 탐색 효율을 개선하는 UI 구조를 설계 및 개선
 - 사용자 탐색 흐름을 고려한 인터페이스 구조 설계
 - 브랜드 아이덴티티를 반영한 UI 디자인 및 퍼블리싱
 - PC → Mobile 반응형 구조 개선 및 사용성 향상

② 데이터 흐름 기반 UI 설계 및 상태 동기화

- ✓ 데이터 상태 변화에 따라 UI가 자동으로 업데이트되는
- ✓ 단일 흐름 기반 구조 설계
 - JSON 데이터 기반 UI 렌더링 및 필터 / 정렬 로직 구현
 - 비동기 처리(fetch)를 통한 데이터 흐름 제어 경험
 - 상태 변화에 따라 UI가 동적으로 업데이트되는 구조 설계
 - UI 상태와 데이터 상태를 일관되게 유지하는 동기화 구조 구현

③ 사용자 인터랙션 기반 상태 관리

- ✓ 사용자 액션에 따라 UI가 즉각 반응하는 인터랙션 중심 구조 설계
- ✓ 사용자 액션 → 상태 → UI로 이어지는 흐름을 설계하여
- ✓ 일관된 인터랙션 경험 제공
 - 필터, 정렬, 탭, 드롭다운 등 사용자 인터랙션 구현
 - localStorage / cookie 기반 사용자 상태 유지
 - 사용자 행동에 따른 UI 상태 변화 및 동적 렌더링 처리
 - UI 상태와 데이터 흐름을 연결하는 구조 설계 경험

④ 지도 기반 서비스 설계 및 구현

- ✓ 위치 데이터를 기반으로 탐색 UX를 개선한 지도 서비스 개발 경험
- ✓ 기존 리스트 중심 탐색을 위치 기반 탐색으로 전환한 경험
 - Kakao Maps API 기반 위치 중심 서비스 구현
 - 지도 bounds 기반 데이터 필터링 및 거리 계산(Haversine) 적용
 - 마커 그룹핑 및 대표 데이터 설정으로 UI 최적화
 - 지도 + 리스트 + 패널 연동 인터랙션 설계

⑤ 실무 협업 및 문제 해결 능력

- ✓ 기획이 없는 상황에서도 문제를 정의하고 해결 방향을 설계하는 능력
- ✓ 불명확한 요구사항을 구조화하여 서비스 형태로 구체화하는 능력
 - ASP 환경을 고려한 프론트엔드 구조 설계
 - 요구사항이 불명확한 상황에서 기능 정의 및 UX 설계 경험
 - 백엔드 연동을 고려한 데이터 구조 및 인터페이스 설계
 - 데이터 → UI 흐름 중심으로 문제를 해결하는 접근 방식

Projects

[MAIN PROJECT]

데이터 흐름과 UX를 기반으로 실제 서비스 문제를 해결한 프로젝트 경험

① 지도 기반 채용 서비스 (AptJob Map)

→ 위치 기반 탐색 UX 개선 및 데이터 흐름 중심 상태 동기화 구조 설계

✓ 기존 리스트 기반 탐색의 한계를 개선하여 위치 중심 탐색 UX로 전환

✓ 지도(bounds) + 사용자 상태 기반 데이터 필터링 구조 설계

✓ 지도 · 리스트 · 패널 UI를 하나의 흐름으로 연결하여 상태 불일치 문제 해결

→ 위치 기반 탐색으로 전환하여 사용자가 원하는 공고를 더 빠르게 찾을 수 있도록 UX 개선

[OTHER PROJECTS]

✓ AptJob 반응형 리디자인

→ PC 중심 구조를 모바일 중심 UX로 개선

→ 카드형 UI 및 필터 UX 단순화

✓ UI Builder

→ 데이터 기반 UI 생성 구조 설계 경험

✓ 대시보드

→ 실시간 데이터 기반 UI 렌더링 및 상태 관리 구현

→ 데이터 시각화를 통해 핵심 정보를 한눈에 파악할 수 있도록 개선

1) [잡코리아] 채용공고 UI Builder (간편 에디터)

✓ 채용 담당자가 직접 공고를 생성할 수 있는 UI Builder 설계 및 구현

✓ 템플릿 기반 입력 구조로 비개발자도 쉽게 사용할 수 있도록 개선

✓ 입력 → 데이터 → UI로 이어지는 구조 설계

✓ 비개발자도 공고를 생성할 수 있도록 UX 단순화

→ 사용성 개선 및 콘텐츠 생성 효율 향상

2) [잡코리아] 영업 실적 대시보드 (CTI 시스템)

✓ 영업 실적(콜 수, 매출, 응답률) 데이터를 시각화하는 대시보드 UI 구현

✓ 데이터 기반 UI 렌더링 구조 설계 및 상태 관리 구현

✓ 그래프 중심 UI로 정보 가독성 개선

✓ 분산된 데이터를 시각화하여 한 화면에서 확인 가능하도록 개선

→ 실시간 데이터 시각화를 통한 의사결정 지원

3) [잡코리아] 브랜드 채용공고 디자인 고도화

✓ 브랜드 아이덴티티를 반영한 채용공고 UI 설계 및 퍼블리싱

✓ 사용자 유입 및 전환율 향상을 위한 UI 개선

✓ 인터랙션 요소 추가 및 유지보수 구조 개선

✓ 사용자 유입 및 지원 전환을 고려한 UI 개선

→ 사용자 경험 개선 및 서비스 품질 향상

4) [잡코리아] 프로모션 및 디자인 시스템 구축

✓ 분기별 채용 프로모션 UI 설계 및 퍼블리싱

✓ 반복 작업을 줄이기 위한 UI 컴포넌트 구조 설계

✓ 브랜드 일관성을 유지하는 디자인 시스템 적용

✓ 반복 작업 감소 및 유지보수 효율 향상

→ 유지보수 효율 향상 및 UI 일관성 확보

신규 기능 기획 및 개발 | 지도 기반 채용 서비스 (AptJob Map)

내 역할(Role)

- 지도 기반 채용 서비스 프론트엔드 전체 설계 및 구현
- Kakao Maps API 연동 및 데이터 시각화 로직 개발
- 마커, 필터, 패널 UI 인터랙션 설계 및 구현
- JSON 기반 데이터 처리 및 상태 관리 로직 구현
- ASP 백엔드 연동을 고려한 데이터 구조 설계

기여도 (Contribution)

- Front-End 100% 담당
- 기획 일부 참여 (UI/UX 구조 설계)
- 데이터 구조 및 인터랙션 설계 주도

작업 기간

- 총 9일 (기획 5일 / 개발 4일)
 - 기획 참여 기반으로 UI/UX 설계 및 프론트엔드 개발 전 과정 수행
 - 지도 기반 인터랙션 및 데이터 흐름 구조 직접 설계

Live Demo 바로가기

1. 문제 정의 (WHY)

기존 서비스에는 위치 기반 공고 탐색 기능이 없어
사용자가 주변 공고를 직관적으로 확인하기 어려웠습니다.

2. 해결 방식 (HOW)

지도 기반 인터페이스를 구축하여
공고 데이터를 위치 중심으로 시각화하고
사용자 액션에 따라 데이터가 실시간 반영되도록 구현했습니다.

- 지도 bounds 기반 데이터 필터링
- 사용자 위치 기반 거리 계산 (Haversine)
- 카테고리 필터 기반 동적 데이터 처리

3. 핵심 기능 (WHAT)

- ✓ 지도 bounds 기반 데이터 필터링
- ✓ 마커 그룹핑 및 대표 공고 설정
- ✓ 거리 계산 (Haversine)
- ✓ 마커 클릭 → 사이드 패널 UI 연동
- ✓ 필터 상태 유지 (localStorage)
- ✓ 지도 + 리스트 + 패널 상태 동기화

4. 기술 포인트 (DEEP DIVE)

- 지도 상태(bounds)와 사용자 필터 상태를 결합한 데이터 필터링 구조 설계
- 동일 좌표 데이터 그룹핑 및 대표 마커 처리 로직 구현
- 지도 이동 시 불필요한 데이터 렌더링 최소화 (성능 고려)
- 지도 / 리스트 / 패널 UI 상태 동기화 구조 설계

신규 기능 기획 및 개발 | 지도 기반 채용 서비스 (AptJob Map)

문제 해결을 위한 서비스 기획 (Planning)

1) 사용자 문제

- 기존 리스트 기반 탐색은 위치 정보를 직관적으로 파악하기 어려움
- 사용자가 "내 주변 공고"를 빠르게 찾기 어려운 구조
- 공고 간 거리 비교 및 지역 분포 확인이 불가능

2) 해결 아이디어

- 지도 기반 UI를 통해 공고를 위치 중심으로 시각화
- 사용자 위치 기준 거리 기반 정렬 기능 제공
- 카테고리 필터를 통한 직무별 탐색 기능 추가

3) UX 설계 핵심

- 지도 / 리스트 / 패널을 하나의 흐름으로 연결
- 사용자 액션(이동, 클릭, 필터)에 따라 UI가 동시에 반응하도록 설계
- 클릭 → 정보 확인 → 상세 이동까지 자연스러운 탐색 흐름 구성
- 필터 상태 유지로 반복 탐색 UX 개선

→ 기존 서비스에 없던 기능을 사용자 경험 개선 관점에서 직접 기획 및 설계



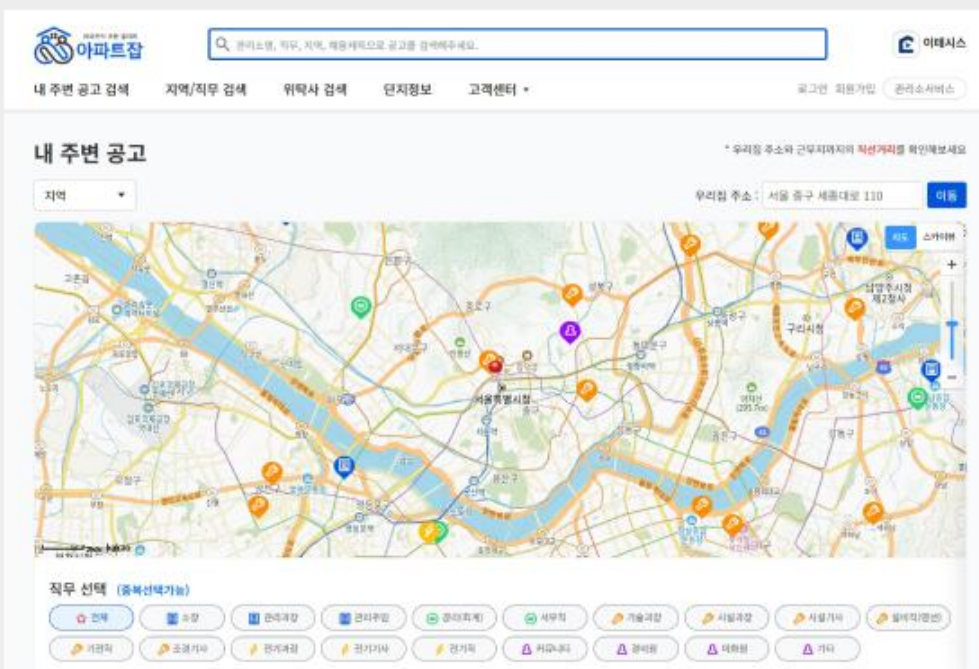
신규 기능 기획 및 개발 | 지도 기반 채용 서비스 (AptJob Map)

Live Demo 바로가기

1. 데이터 시각화

광고 데이터를 단순 리스트가 아닌 지도 위 마커 형태로 시각화하여 사용자가 공간적으로 광고 분포를 직관적으로 파악할 수 있도록 구현했습니다.

각 광고는 좌표(lat, lng)를 기준으로 마커로 생성되며, 마커 아이콘은 직무 카테고리에 따라 다르게 표시되도록 설계했습니다. 이를 통해 사용자는 특정 지역의 광고 밀집도와 직무 분포를 한눈에 파악할 수 있습니다.



전체 광고 총 25건

광고제목	직무	급여	단지명	단지규모	등록일	마감일	거리
서울_기안직_경력	기안직	월급3,230,000	경희궁역(마포4단지	120세대	2026-03-23	2026-04-10	0.8 km

2. 데이터 필터링 (bounds)

지도 이동 및 확대/축소 시 현재 화면에 보이는 영역(bounds)을 기준으로 광고 데이터를 실시간 필터링하도록 구현했습니다.

지도 객체의 bounds.contains() 메서드를 활용하여 각 광고의 좌표가 현재 화면 내에 포함되는지 판단하고, 해당 광고만 렌더링하도록 처리했습니다. 이를 통해 불필요한 데이터 노출을 줄이고 사용자가 현재 보고 있는 지역에 집중할 수 있도록 UX를 개선했습니다.

3. 거리 계산 (Haversine)

사용자의 현재 위치를 기준으로 각 광고까지의 거리를 계산하여 거리 순으로 정렬할 수 있도록 구현했습니다. 거리 계산은 Haversine 공식을 사용하여 두 좌표 간의 실제 지구 거리(km)를 계산하도록 설계했습니다. 이를 통해 사용자는 가까운 광고를 우선적으로 확인할 수 있으며, 실제 사용 환경에 맞는 탐색 경험을 제공합니다.

4. 데이터 그룹핑

동일한 아파트(같은 좌표)에 여러 개의 광고가 존재하는 경우, 중복 마커가 생성되는 문제를 해결하기 위해 그룹핑 로직을 구현했습니다. 같은 lat/lng를 기준으로 데이터를 묶고, 가장 최신 광고를 대표 마커로 설정하여 표시했습니다. 마커 클릭 시 해당 위치의 전체 광고 리스트를 사이드 패널에서 확인할 수 있도록 구성했습니다.

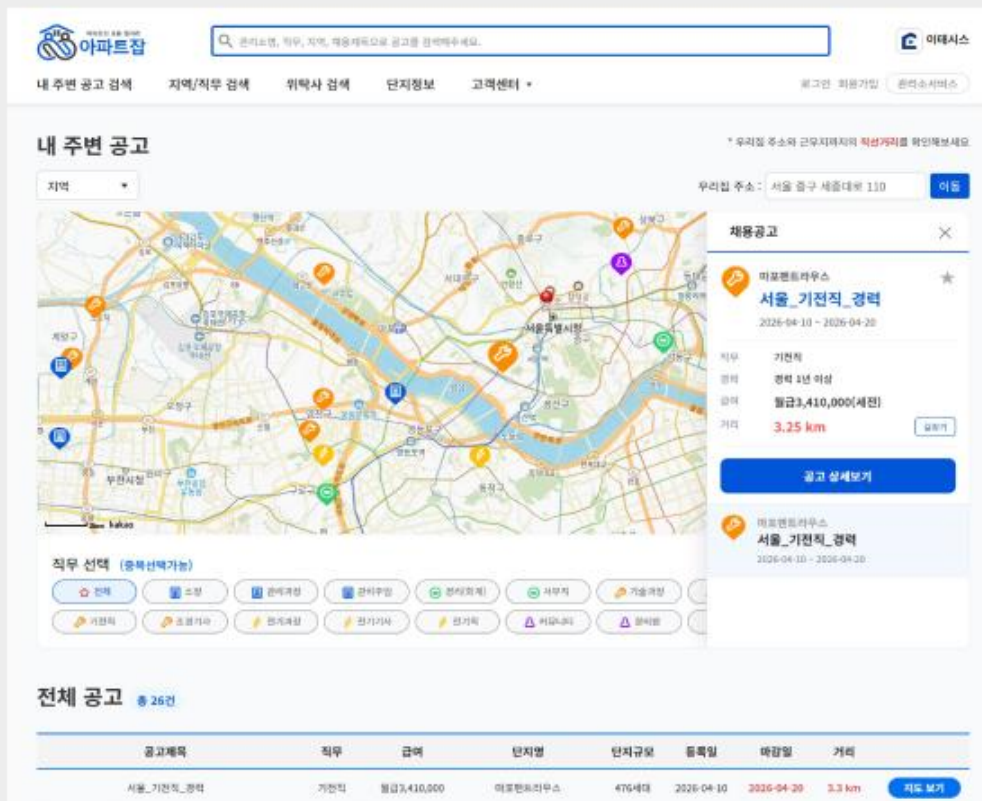
신규 기능 기획 및 개발 | 지도 기반 채용 서비스 (AptJob Map)

Live Demo 바로가기

5. 마커 인터랙션 & 패널 UI

사용자가 마커를 클릭하면 해당 위치의 공고 정보를 우측 사이드 패널에 표시하도록 구현했습니다. 패널에는 대표 공고 요약 정보와 함께 동일 위치의 다른 공고 리스트를 함께 제공하여 사용자가 추가 탐색을 할 수 있도록 구성했습니다. 또한 패널 내 공고 클릭 시 상세 페이지로 이동할 수 있도록 인터랙션 흐름을 설계했습니다.

→ 사용자 액션에 따라 UI가 즉시 반응하도록 설계



6. 카테고리 필터 (다중 선택)

직무별 공고를 필터링할 수 있도록 다중 선택 기반 카테고리 필터 기능을 구현했습니다. 선택된 카테고리는 Set 자료구조로 관리하여 중복 없이 상태를 유지하도록 설계했으며, 필터 변경 시 지도, 마커, 리스트가 동시에 갱신되도록 처리했습니다. '전체' 선택 시 모든 데이터를 초기화하는 예외 처리도 함께 구현했습니다.

7. 사용자 상태 유지 (localStorage / cookie)

사용자가 선택한 직무 필터 및 위치 정보는 localStorage와 cookie를 활용하여 저장되도록 구현했습니다.

페이지 재접속 시 이전 상태를 복원하여 사용자가 반복적으로 설정할 필요 없이 연속적인 탐색 경험을 유지할 수 있도록 설계했습니다.

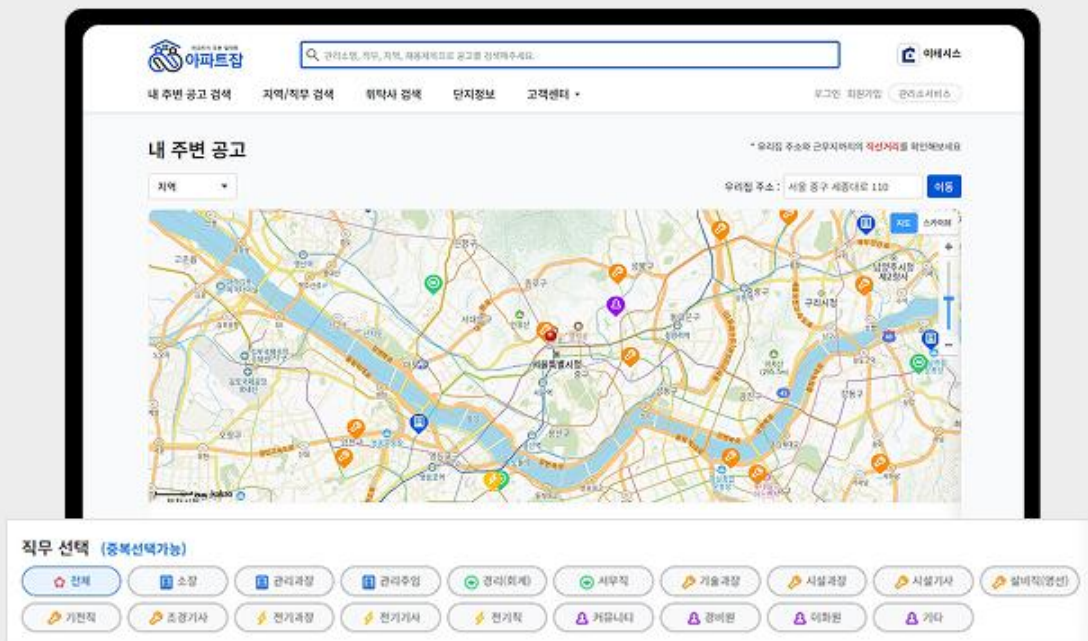
8. 지도 + 리스트 + 패널 상태 동기화

지도 이동, 필터 변경, 마커 클릭 등 다양한 사용자 액션에 따라 지도, 리스트, 패널 UI가 동시에 업데이트되도록 상태 동기화 구조를 설계했습니다.

단일 상태 흐름을 기준으로 데이터 → 마커 → 리스트 → 패널 순으로 렌더링되도록 구성하여 UI 간 불일치 문제를 해결했습니다.

신규 기능 기획 및 개발 | 지도 기반 채용 서비스 (AptJob Map)

Live Demo 바로가기



지도 마커 생성 및 상태 제어 로직

- 선택된 마커(activeMarker) 상태 관리
- 기존 선택 마커 초기화 → 새 마커 활성화
- 활성 마커 강조 (크기 + zIndex)
- 클릭 시 지도 중심 이동(panTo)

→ 사용자 인터랙션 흐름에 맞춰 상태를 제어하는 구조

마커 필터링 및 동적 업데이트

- 선택된 직무 기준 마커 필터링
- 조건에 맞지 않는 마커 제거
- 지도 상태와 데이터 상태 동기화

→ UI와 데이터 상태를 연결하는 핵심 로직

```
let activeMarker = null;
function setActiveMarker(marker) {
  if (activeMarker) {
    activeMarker.setImage(createMarkerImage(activeMarker.category, false));
    activeMarker.setZIndex(1); }

  activeMarker = marker;

  marker.setImage(createMarkerImage(marker.category, true));
  marker.setZIndex(999);

  map.panTo(marker.getPosition());
}

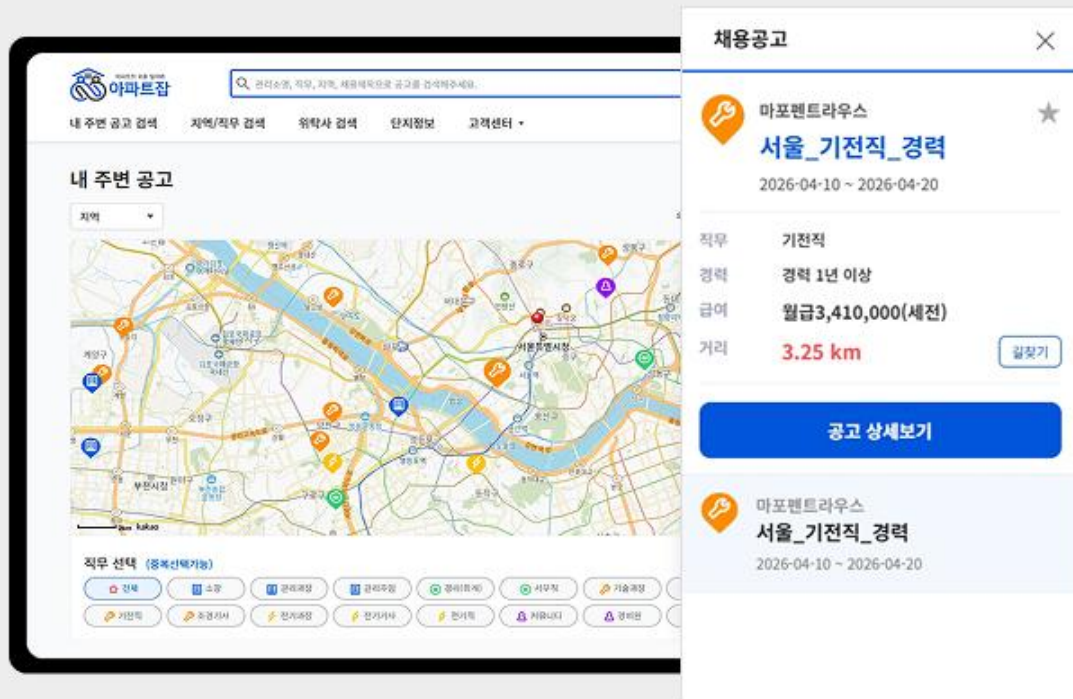
function updateMarkers() {
  markers.forEach(marker => {
    const filteredJobs = marker.jobs.filter(job =>
      selectedJobs.has(job.jobType));

    if (filteredJobs.length === 0) {
      marker.setMap(null);
      return; }

    marker.setMap(map);
  });
}
```

신규 기능 기획 및 개발 | 지도 기반 채용 서비스 (AptJob Map)

Live Demo 바로가기



공고 패널 UI 생성 및 데이터 처리

- 동일 위치 공고 데이터를 그룹 단위로 관리
- 최신 공고 기준 정렬 후 대표 데이터 선택
- 패널 UI에 동적 렌더링

→ 데이터 중심 UI 구조 설계

패널 ↔ 마커 상태 동기화

- 패널에서 선택한 공고 기준으로 마커 위치 변경
- 활성 마커 UI 업데이트
- 지도 중심 이동

→ 리스트 → 지도까지 연결된 인터랙션 구현

```
function openPanel(key) {
  const jobs = groupedPositions[key];

  const sorted = [...jobs].sort(
    (a, b) => new Date(b.startDate) - new Date(a.startDate)
  );

  currentSelectedJob = sorted[0];

  document.getElementById("jobPanelContent").innerHTML = `
    ${renderSummary(currentSelectedJob)}
  `;
}

function selectJob(index) {
  const job = currentPanelJobs[index];

  currentSelectedJob = job;

  const marker = markers.find(m => m.groupKey === currentGroupKey);

  marker.setPosition(job.latlng);
  marker.setImage(createMarkerImage(getCategoryGroup(job.jobType), true));

  map.panTo(job.latlng);
  activeMarker = marker;
}
```

신규 기능 기획 및 개발 | 지도 기반 채용 서비스 (AptJob Map)

Live Demo 바로가기

성과 및 임팩트 (Impact)

- ✓ 지도 기반 UI 도입으로 리스트 중심 탐색 → 위치 기반 탐색으로 전환
- ✓ 위치/거리 기반 정보 제공으로 사용자 의사결정 향상
- ✓ 상태 동기화 구조 설계를 통해 지도 / 리스트 / 패널 간 UI 불일치 문제 해결
- ✓ 데이터 중심 UI 설계를 통한 사용자 경험 개선

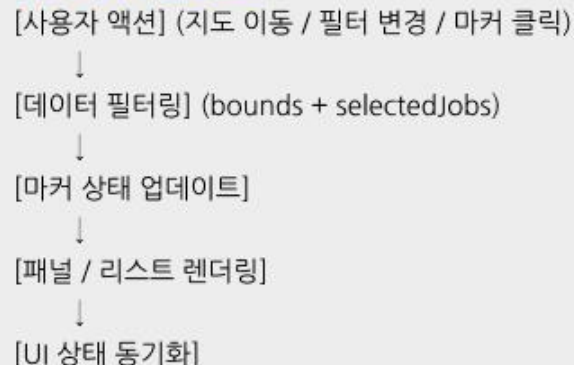
코드 설명 한 줄 강화

- 1) 마커 상태 관리 코드
: 사용자 클릭 상태(activeMarker)를 기준으로 마커 UI를 동적으로 변경하여 인터랙션 흐름을 제어했습니다.
- 2) 필터링 코드
: 지도 bounds와 사용자 선택 상태를 기준으로 데이터를 필터링하여 UI가 실시간으로 반영되도록 설계했습니다.
- 3) 패널 연동 코드
: 패널에서 선택한 데이터가 마커와 지도에 즉시 반영되도록 UI 간 상태를 하나의 흐름으로 연결했습니다.
- 4) 상태 동기화 코드 (핵심🔥)
: 지도, 리스트, 패널이 각각 동작하지 않도록 데이터 중심 단일 상태 흐름 구조로 설계했습니다.

기술 스택 요약

- HTML / CSS
- JavaScript (ES6+)
- Kakao Maps API
- LocalStorage / Cookie
- JSON Data Handling
- ASP 환경 연동을 고려한 데이터 구조 설계 경험

아키텍처 흐름도



문제 해결 (Trouble Shooting)

- 문제
: 지도 이동 시 마커, 리스트, 패널 UI가 서로 다른 상태를 표시
- 원인
: 각 UI가 독립적으로 상태를 관리하고 있었음
- 해결
: 데이터 → UI 단일 흐름 구조로 재설계
- 개선 효과
: 사용자 액션에 따른 UI 반응 일관성 확보 및 유지보수 용이성 향상

지도 기반 공고 탐색 시스템 설계 (Map Interaction & Data Handling)

Live Demo 바로가기

1. 문제 정의

공고가 많아질수록 사용자가 원하는 채용 정보를 빠르게 찾기 어려움

- 리스트 기반 탐색의 한계
- 위치 기반 정보 파악 어려움
- 동일 위치 다수 공고 처리 문제

3. 핵심 구조

/assets/js/map

- map-core.js (지도 초기화)
- map-data.js (데이터 관리)
- map-marker.js (마커 생성/제어)
- map-filter.js (카테고리 필터)
- map-panel.js (패널 UI 렌더링)

: 기능 단위로 JS 모듈 분리하여 유지보수성과 확장성 고려

5. 성과 / 차별점

- ✓ 사용자 위치 기반 공고 탐색 UX 개선
- ✓ 지도 + 리스트 동기화 구현
- ✓ 동일 위치 공고 그룹핑 처리
- ✓ 실시간 필터링으로 탐색 속도 향상
- ✓ 사용자 중심 인터랙션 설계

: UI 구현을 넘어, 데이터 흐름과 사용자 경험까지 설계한 지도 기반 서비스

2. 해결 전략

- ✓ 지도 기반 탐색 UI 설계
- ✓ Map Bounds 기반 실시간 필터링
- ✓ 직무 카테고리 다중 선택 필터
- ✓ 동일 위치 공고 그룹핑 처리
- ✓ 거리 기반 정렬 (사용자 중심 UX)

4. 핵심 로직

```
// 지도 영역 필터링
const visibleJobs = jobs.filter(job => bounds.contains(job.latlng));

// 카테고리 필터 (다중 선택)
const selected = new Set();
const toggleCategory = c =>
  selected.has(c) ? selected.delete(c) : selected.add(c);

// 거리 계산 (Haversine)
const getDistance = (a, b, c, d) => {
  const R = 6371, toRad = x => x * Math.PI / 180;
  const dLat = toRad(c - a), dLng = toRad(d - b);
  const v =
    Math.sin(dLat/2)**2 +
    Math.cos(toRad(a)) * Math.cos(toRad(c)) *
    Math.sin(dLng/2)**2;
  return R * (2 * Math.atan2(Math.sqrt(v), Math.sqrt(1 - v)));
};
```

UX 개선을 위한 신규 기능 기획 및 반응형 웹 개발

| AptJob 반응형 리디자인

1. 프로젝트 개요

- 프로젝트명 : AptJob 반응형 리디자인
- 작업 기간 : 2026.02 ~ 계속 업데이트 중
- 기여도 : 100% (UI/UX 설계 + FE 개발)
- 배포 링크 : <https://github.com/naseri3/aptjob-responsive.git>

2. 프로젝트 목적

: 기존 PC 중심 구조에서 발생하는 사용자 불편을 해결하고
모바일 환경에서도 편리하게 사용할 수 있는 채용 서비스로 개선

3. 기존 문제 (Before)

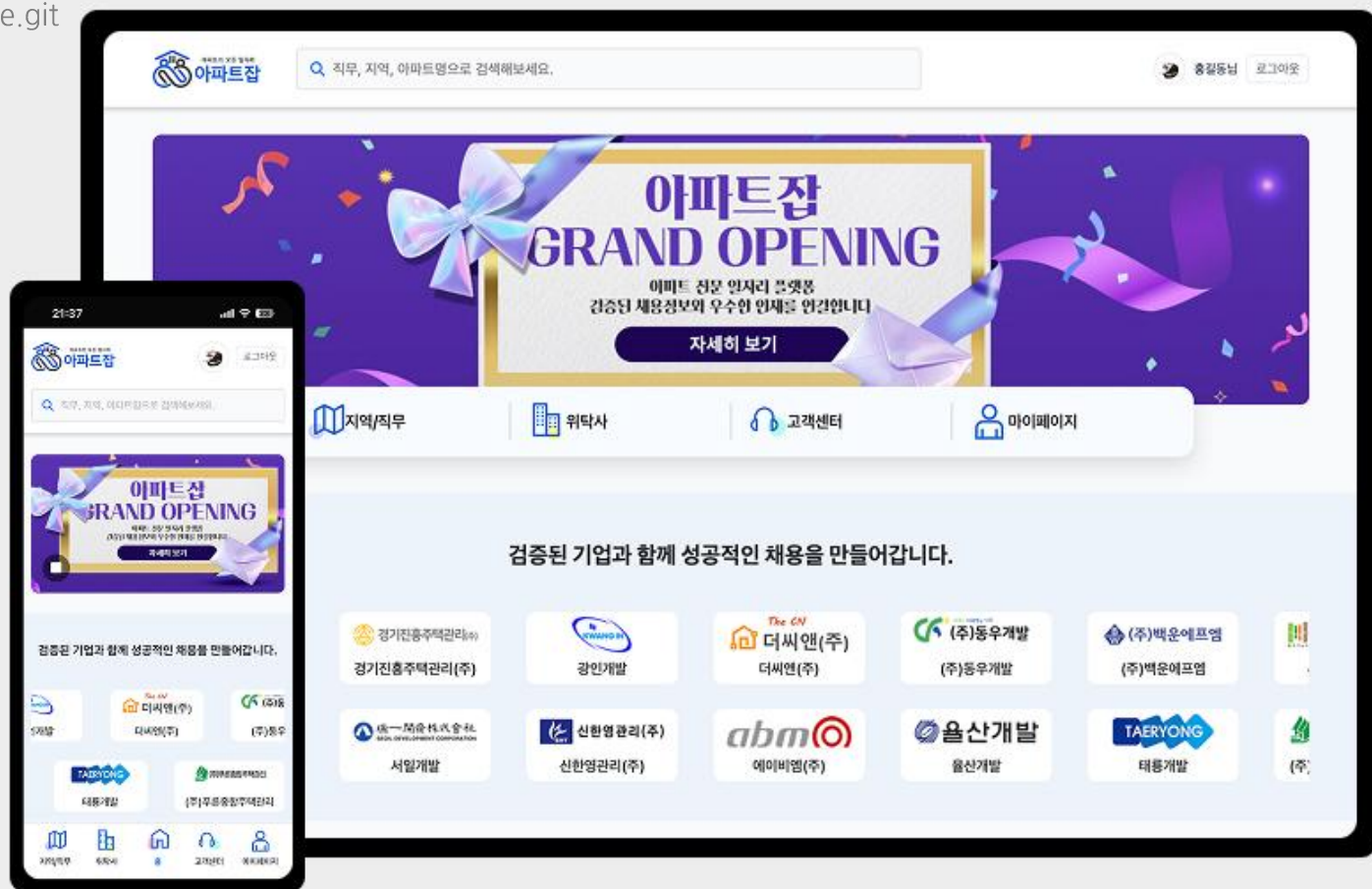
- PC 전용 UI → 모바일 사용성 부족
- 정보 과다 → 가독성 저하
- 검색 / 필터 UX 불편
- CTA(지원 버튼) 시인성 낮음

4. 개선 목표 (After)

- 모바일 중심 반응형 UI 설계
- 카드형 UI로 정보 구조 개선
- 검색 / 필터 UX 단순화
- CTA 버튼 강조로 지원 유도

5. 사용 기술

- HTML / SCSS / Bootstrap
- Vanilla JavaScript
- Netlify 배포

[GitHub 코드 보기](#)[Live Demo 바로가기](#)

UX 개선을 위한 신규 기능 기획 및 반응형 웹 개발

| AptJob 반응형 리디자인

[GitHub 코드 보기](#)[Live Demo 바로가기](#)

1. 정보 과다 & 가독성 문제

- 문제 상황
- 채용 공고가 텍스트 리스트 형태로 나열
 - 중요 정보(지역, 연봉, 상태)가 한 줄에 혼재
 - 시선 흐름이 분산됨

2. 모바일 UX 미지원

- 문제 상황
- PC 레이아웃 그대로 축소
 - 클릭 영역 작음
 - 가독성 붕괴

3. 검색 / 필터 UX 복잡

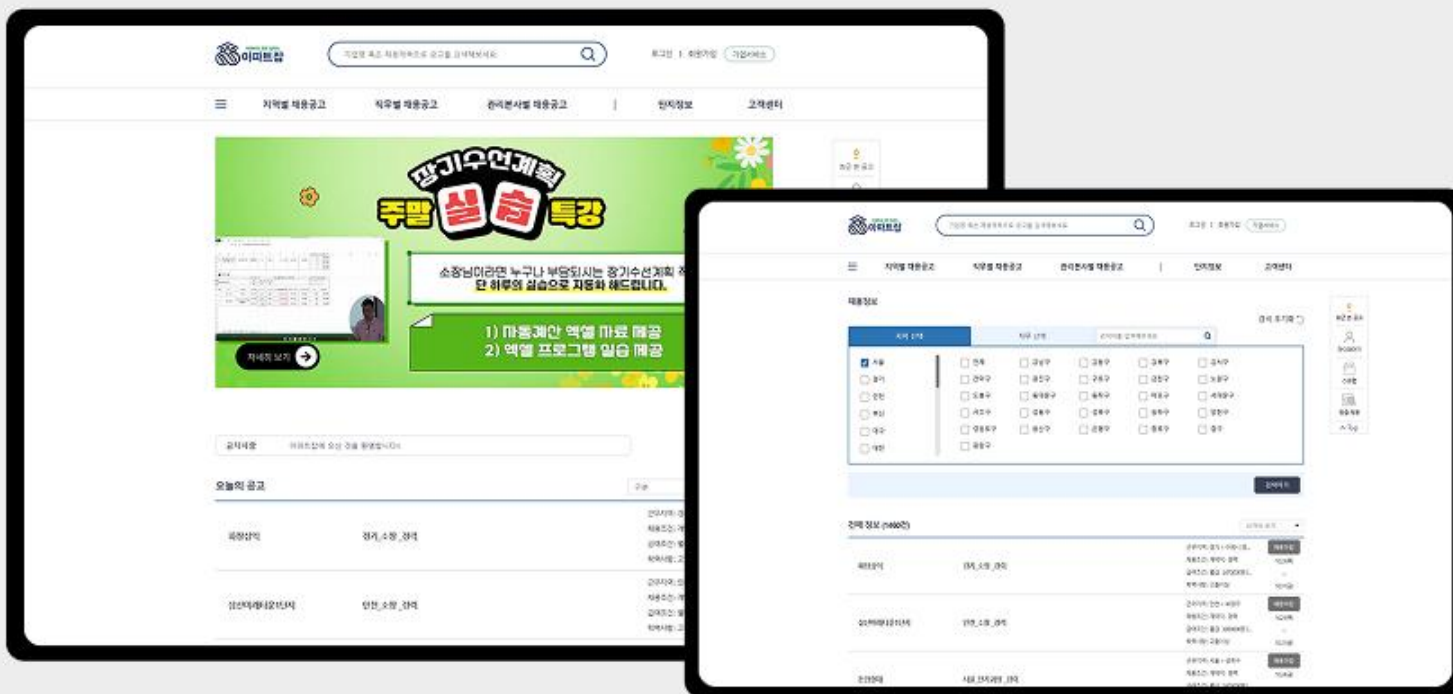
- 문제 상황
- 지역 / 직무 선택 UI 분리
 - 필터 구조 직관적이지 않음
 - 선택 상태 인지 어려움

4. CTA (지원 유도) 약함

- 문제 상황
- "채용중" 버튼이 정보와 동일한 레벨
 - 강조 부족

5. UI 일관성 부족

- 문제 상황
- 페이지별 UI 구조 상이 (공지 / 기업 / 문의 등)
 - 동일 기능인데 디자인 다름



UX 개선을 위한 신규 기능 기획 및 반응형 웹 개발

| AptJob 반응형 리디자인

1. 카드형 UI 구조 적용

- 텍스트 리스트 → 카드형 UI로 전환
- 주요 정보(지역, 직무, 연봉) 시각적으로 분리

☞ ✓ 스캔 속도 증가

☞ ✓ 가독성 개선

2. 모바일 반응형 UX 적용

- Bootstrap Grid 기반 반응형 설계
- 모바일에서 세로 스크롤 중심 UX

☞ ✓ 터치 영역 확대

☞ ✓ 모바일 사용성 개선

3. 검색 / 필터 UX 개선

- 지역 / 직무 선택 UI 통합
- 선택 상태 시각화 (active 상태)

☞ ✓ 탐색 흐름 자연스럽게 연결

☞ ✓ 사용자 피로도 감소

4. CTA 강조 구조 개선

- “채용중” → 버튼 강조 스타일 적용
- 주요 액션 영역 시각적 분리

☞ ✓ 클릭 유도 증가

☞ ✓ 전환 UX 개선

5. UI 일관성 정리

- 공통 레이아웃 구조 통일
- 동일 기능 → 동일 UI 패턴 적용

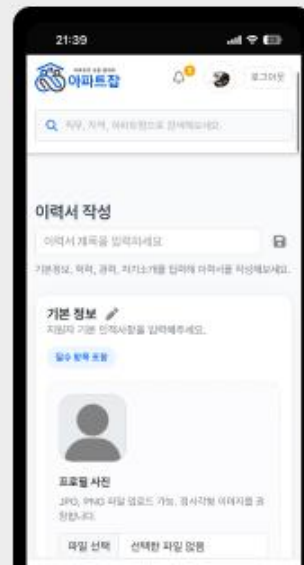
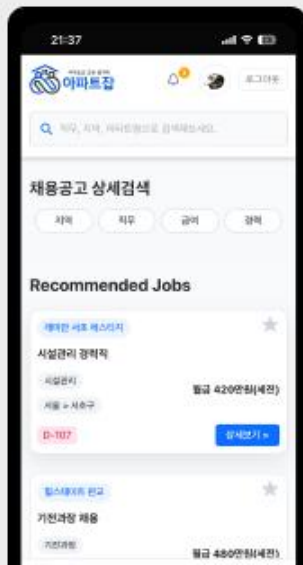
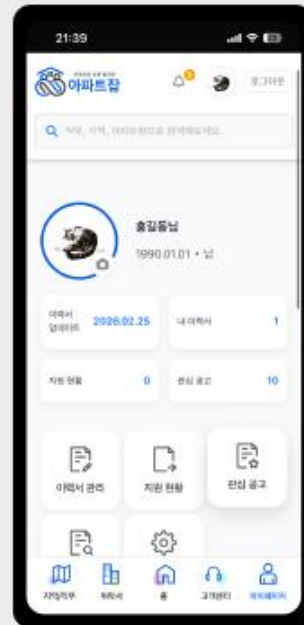
☞ ✓ 사용자 학습 비용 감소

☞ ✓ 전체 UX 완성도 향상

사용자가 “빠르게 찾고 → 바로 지원”할 수 있는 UX 구조로 개선

GitHub 코드 보기

Live Demo 바로가기



UX 개선을 위한 신규 기능 기획 및 반응형 웹 개발

| AptJob 반응형 리디자인

Before vs After

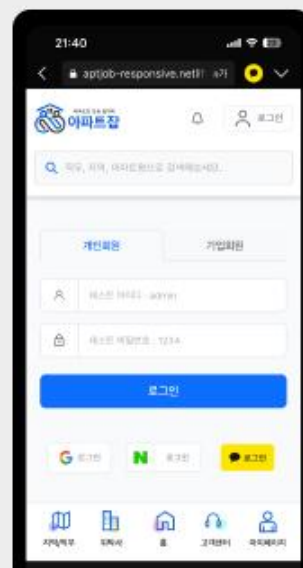
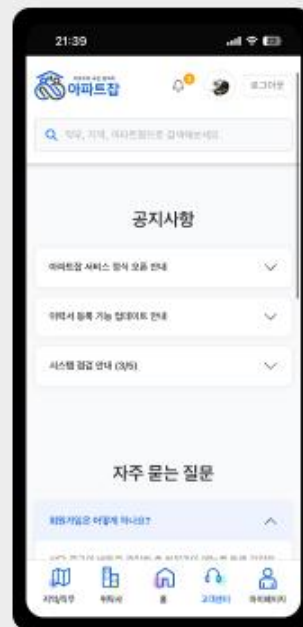
구분	Before	After
구조	텍스트 리스트	카드형 UI
가독성	정보 밀집	정보 분리
모바일	미지원	반응형 지원
필터UX	복잡	직관적
CTA	약함	강조
UX흐름	끊김	자연스러움

구현 포인트

- Bootstrap Grid 기반 반응형 레이아웃 설계
- SCSS 모듈화로 UI 컴포넌트 구조 개선
- JavaScript로 필터 상태 관리 및 UI 동기화 구현

GitHub 코드 보기

Live Demo 바로가기



UX 개선을 위한 신규 기능 기획 및 반응형 웹 개발

| AptJob 반응형 리디자인

[GitHub 코드 보기](#)[Live Demo 바로가기](#)

기술 구현 상세

- Bootstrap Grid 기반 반응형 UI 설계
- SCSS를 활용한 컴포넌트 단위 스타일 구조 설계
- JavaScript(Set)을 활용한 필터 상태 관리 구현
- 데이터 기반 렌더링 방식으로 UI 동적 생성
- 필터 → 데이터 → UI까지 연결된 구조 설계
- 사용자 상태(active) 시각화 처리

개발 구조 및 핵심 로직

사용자 액션 (필터 선택 / 검색 입력)



데이터 상태 관리 (Set / Array)



데이터 필터링 (filter / map)



UI 렌더링 (카드 / 리스트)



UI 상태 동기화 (active / count)

1. 데이터 기반 UI 렌더링

- JSON 데이터 기반으로 공고 리스트를 동적으로 렌더링
- map()을 활용하여 UI 컴포넌트 생성 구조 설계
- 데이터 변경 시 UI가 자동으로 업데이트되는 구조 구현

2. 필터 상태 관리

- Set 자료구조를 활용하여 직무/지역 필터 상태 관리
- 중복 없이 선택 상태 유지
- 필터 변경 시 데이터와 UI 동기화 처리

3. UI 상태 동기화

- 필터 선택 상태(active UI)와 데이터 상태를 동기화
- 선택/해제 시 UI와 리스트가 동시에 업데이트되도록 설계
- 사용자 액션 기반 즉각적인 UI 반응 처리

4. 반응형 UI 구조 설계

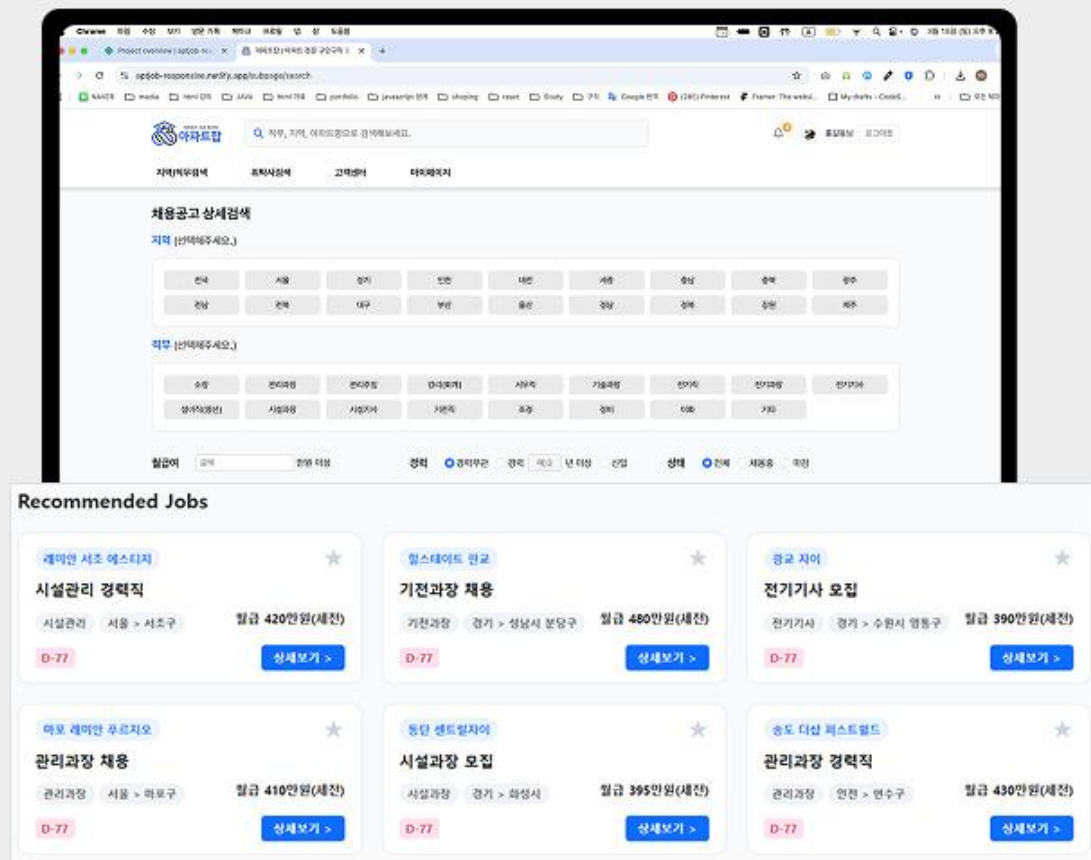
- Bootstrap Grid 기반 반응형 레이아웃 설계
- PC / Mobile UI 구조 분리 및 최적화
- 터치 환경을 고려한 인터랙션 개선

5. 사용자 상태 관리

- localStorage를 활용한 사용자 상태 유지 (필터, 관심 공고 등)
- 페이지 이동 후에도 사용자 선택 상태 유지

UX 개선을 위한 신규 기능 기획 및 반응형 웹 개발

| AptJob 반응형 리디자인

[GitHub 코드 보기](#)[Live Demo 바로가기](#)

✓ 필터 상태 관리 (Set)

- 사용자 선택 상태를 Set으로 관리하여 중복 없이 유지

✓ 데이터 필터링

- 선택된 필터 기준으로 공고 데이터 필터링

✓ UI 렌더링

- 필터링된 데이터를 기반으로 리스트 재생성

✓ 상태 동기화

- 필터 상태와 UI(active, count)를 동시에 업데이트

```
// 선택된 직무 상태 관리
```

```
const selectedJobs = new Set();
```

```
// 필터 클릭 이벤트
```

```
function toggleJobFilter(jobType) {
```

```
  if (selectedJobs.has(jobType)) {
```

```
    selectedJobs.delete(jobType);
```

```
  } else {
```

```
    selectedJobs.add(jobType);
```

```
  }
```

```
  updateJobList();
```

```
}
```

```
// 데이터 필터링 + UI 렌더링
```

```
function updateJobList() {
```

```
  const filteredJobs = jobs.filter(job =>
```

```
    selectedJobs.size === 0 || selectedJobs.has(job.type)
```

```
  );
```

```
  renderJobList(filteredJobs);
```

```
}
```

사용자 필터 상태(Set)를 기준으로 데이터를 필터링하고,
필터링된 결과를 UI에 즉시 반영하도록 설계

UX 개선을 위한 신규 기능 기획 및 반응형 웹 개발

| AptJob 반응형 리디자인

[GitHub 코드 보기](#)[Live Demo 바로가기](#)

실무 문제 해결 경험

문제 : 지도 이동 시 마커 / 리스트 / 패널 UI가 서로 다른 상태를 표시

원인 : UI별로 상태를 각각 관리하고 있었음

해결

- 데이터 중심 단일 상태 흐름으로 구조 재설계
- 사용자 액션 → 데이터 → UI 순으로 흐름 통일

결과

- UI 불일치 문제 해결
- 유지보수 구조 개선

아키텍처 설계

[사용자 액션]



[상태 관리 (Set / State)]



[데이터 필터링]



[UI 렌더링]



[상태 동기화]

성능 개선 / 최적화

성능 최적화

- 지도 이동 시 필요한 데이터만 렌더링 (bounds 기반 필터링)
- 불필요한 DOM 업데이트 최소화
- 중복 데이터 그룹핑으로 마커 개수 최적화

결과

- 렌더링 성능 개선
- 사용자 체감 속도 향상

사용자 관점 결과 (Impact 강화)

- 위치 기반 탐색으로 공고 탐색 시간 단축
- 필터 상태 유지로 반복 탐색 UX 개선
- 지도 + 리스트 + 패널 연결로 탐색 흐름 자연화

협업 고려

- ASP 백엔드 연동을 고려한 JSON 데이터 구조 설계
- 공고 마감일 처리 기준 정의 (23:59:59)
- 데이터 배치 처리 방식 제안 (30분 단위 JSON 생성)

UX 개선을 위한 신규 기능 기획 및 반응형 웹 개발

| AptJob 반응형 리디자인

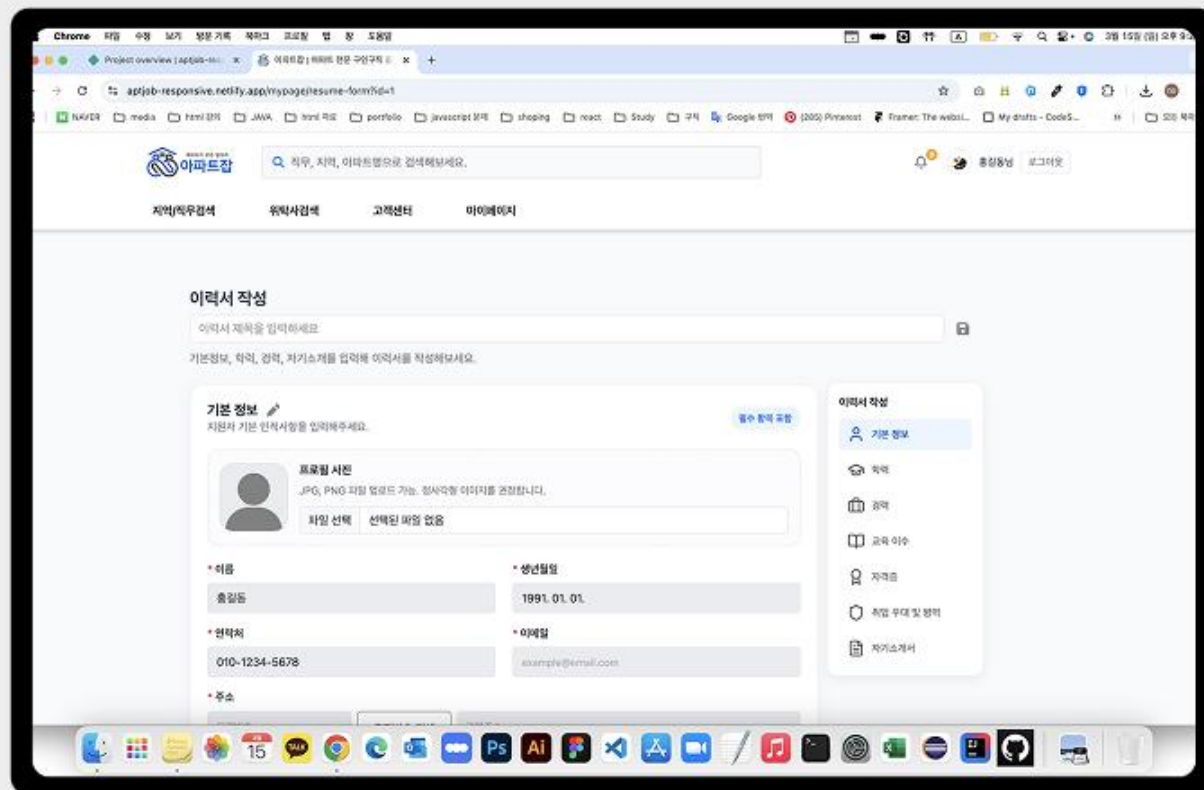
[GitHub 코드 보기](#)[Live Demo 바로가기](#)

기술 선택 이유 (WHY)

- Vanilla JS → 가벼운 구조 및 빠른 렌더링
- Set 자료구조 → 필터 상태 관리 최적화
- LocalStorage → 사용자 경험 유지
- Kakao Maps API → 국내 서비스 최적화

확장 가능성 (미래 설계)

- React 상태 관리 구조로 확장 가능
- 서버 API 연동 시 실시간 데이터 처리 가능
- 사용자 맞춤 추천 기능 추가 가능



감사합니다.

UX 문제를 정의하고 기획 → UI 설계 → 개발까지 수행하는
Front-End Developer 임나슬

Name : 임나슬 Lim Na Seul

E-mail : limnaseul52278@gmail.com

Phone Number : +82) 010-8923-1353
